ALPHA — TRILOGY I OF III · 삼부작의 첫 번째

LLM 시대에 인간의 사고는 어떤 외부 구조를 필요로 하는가

이론에서 시스템으로의 연역적 논증.
LLM의 구조적 불가능에서 출발해,
자기개선 온톨로지의 필연성을 증명한다.

11 Theses

THESIS 0 LLM은 무엇이 구조적으로 불가능한가
할루시네이션·맥락소멸·비구조성은 스케일링으로 해결 안 된다

THESIS 1 토큰에서 의미까지의 세 가지 간극
토큰-의미, 문장-구조, 컨텍스트-세계

THESIS 2 온톨로지의 필연: 왜 벡터 DB로는 안 되는가
벡터 DB의 세 가지 실패 모드 → 하이브리드 검색

THESIS 3 정적 온톨로지도 부족하다
BFO/DOLCE/Gene Ontology가 개인 인지에 안 맞는 이유

THESIS 4 승격: 경험이 원칙이 되는 메커니즘
L0→L5 증류 피라미드, BCM 게이트, DB 분포 실증

THESIS 5 역방향 흐름: 원칙이 경험을 재해석한다
WIM 7렌즈, 들뢰즈 반-현실화

THESIS 6 자기교정: 2차 피드백 루프의 구현
Correction recall 최우선, node #5353/#5369

THESIS 7 세션 주입: 의식의 작업대를 설계한다
작업기억 대응, L2+ 우선, Context as Currency

THESIS 8 에코시스템: 온톨로지 바깥의 온톨로지
12개 프로젝트, foundation/ 자기참조, 인지 순환

THESIS 9 경쟁 지형: 아무도 점유하지 않은 공간
2×2 매트릭스, 우상단 빈 공간

THESIS 10 이것이 Human-AI 협업의 미래에 대해 말하는 것
LLM은 매체, 매체에는 구조, 구조는 살아있어야

THESIS 00

LLM은 무엇이 구조적으로 불가능한가

할루시네이션, 맥락 소멸, 비구조적 지식 — 파라미터 스케일링으로 해결되지 않는 아키텍처적 한계

LLM의 한계를 말할 때 흔히 "아직 부족하다"는 표현을 쓴다. 틀렸다. 문제는 양이 아니라 아키텍처다. 트랜스포머 기반 언어 모델은 토큰 시퀀스의 조건부 확률 분포를 학습한다. 이 구조에서 세 가지는 파라미터를 아무리 늘려도 해결되지 않는다.

> 할루시네이션은 기능이다. 모델은 "가장 그럴듯한 다음 토큰"을 생성한다. "그럴듯함"과 "참"은 다른 기준이다.

확률 분포에는 진위를 판별하는 내부 메커니즘이 없다. Scaling law는 표면적 유창성을 개선하지, 사실성을 보장하지 않는다.

둘째, 맥락은 소멸한다. 200K든 1M이든 컨텍스트 윈도우가 끝나면 모든 것이 사라진다. 대화가 끝나면 관계도, 판단도, 학습도 없다. 이건 메모리 부족이 아니라, 아키텍처에 영속적 상태가 존재하지 않는다는 뜻이다.

셋째, 지식은 비구조적이다. 파라미터에 인코딩된 지식은 flat하다. "A는 B의 원인이다"와 "A는 B와 관련 있다"가 구분되지 않는다. 임베딩 공간에 암묵적으로 녹아 있을 뿐, 명시적으로 질의하거나 검증할 수 없다.

이 세 가지는 "앞으로 해결될 문제"가 아니다. 토큰 확률 모델의 수학적 구조에서 필연적으로 발생하는 속성이다. 실제 사례가 있다. mcp-memory 온톨로지 node #5367은 이렇게 기록한다: "Perplexity sonar-deep-research API가 할루시네이션으로 가짜 뉴스를 게시했다. Google Titan 칩, Vision Pro $499 — URL 0/3 미검증. 검증 로직이 경고만 하고 통과시켰다." LLM 단독 운영의 실패를 직접 겪고 기록한 증거다.

o3/o4 reasoning 모델은 Chain-of-Thought를 내재화해 논리적 일관성을 상당히 개선했고, 1M+ 컨텍스트는 맥락 소멸을 부분적으로 완화한다. 이건 진전이다. 그러나 reasoning 모델도 여전히 토큰 확률로 추론을 생성한다 — "추론 과정이 그럴듯한 것"과 "논리적으로 타당한 것"은 여전히 다르다. 1M 컨텍스트도 세션이 끝나면 사라진다 — 용량이 커졌을 뿐 영속적 상태의 부재는 그대로다. Scaling law는 표면적 유창성과 추론력을 개선하지만, 사실성의 내적 보장이나 영속적 구조를 제공하지 않는다.

STRUCTURAL IMPOSSIBILITIES

HALLUCINATION	P(token) ≈ plausible plausible ≠ true	확률 분포에는 진위 판별 메커니즘이 없다
CONTEXT LOSS	session boundary = erasure no persistent state	200K든 1M이든 세션이 끝나면 소멸
UNSTRUCTURED	token sequence = flat no typed relations	원인-결과와 단순 연관이 구분 불가

THESE ARE NOT "YET-TO-BE-SOLVED" PROBLEMS. THEY ARE ARCHITECTURAL PROPERTIES OF TOKEN-PROBABILITY MODELS.

● SCALING ≠ SOLVING

FIG. 0 — THREE IMPOSSIBILITIES MATRIX

THESIS 01

토큰에서 의미까지의 세 가지 간극

언어학의 세 층위에서 LLM이 어디까지 도달하고 어디서 끊기는지

LLM은 Language Model이다. 그런데 "Language"의 깊이를 대부분 과소평가한다. 언어학의 세 층위에서 LLM이 어디까지 도달하고 어디서 끊기는지를 보면 구조적 결핍의 정확한 위치가 보인다.

> 간극 1: 토큰-의미. BPE는 통계적으로 자주 등장하는 바이트 쌍을 병합한다. 토크나이저는 의미를 이해해서 분리하는 게 아니라 빈도를 세서 분리한다.

"unhappiness"가 *un + happi + ness*로 분리되는 건 형태소 분석의 근사치다. 그러나 근사치일 뿐이다. 의미적으로 중요한 드문 단어는 잘게 쪼개지고, 의미 없이 자주 등장하는 패턴은 하나로 뭉쳐진다.

> 간극 2: 문장-구조. 트랜스포머의 어텐션은 통사론적 의존성의 근사치다. 논리적 의존성은 의미론의 영역이며, 어텐션은 이걸 보장하지 않는다.

주어-동사 일치, 관계절 바인딩까지는 잘 잡는다. 그러나 "A이면 B이고, B이면 C이므로 A이면 C다"는 통사론이 아니라 의미론의 영역이다. CoT와 ToT가 이 간극을 좁히고 있다는 것은 인정해야 한다 — 중간 추론 단계를 명시적으로 생성해 단일 세션 내 논리적 일관성을 높인다. 그럼에도 구조가 필요한 이유: CoT는 세션 내 추론을 개선할 뿐, 세션 간 추론의 연속성을 보장하지 않는다. "지난 주에 A를 결정했고, 전제 B가 틀렸으면 A를 재검토해야 한다" — 이런 장기적 논리 추적은 CoT 범위 밖이다.

> 간극 3: 컨텍스트-세계. 프롬프트 엔지니어링은 화용론의 실천이다. 그러나 화용론은 세계 모델을 전제한다. LLM에는 이 세계 모델이 없다.

이 세 간극은 각각 다른 보완 메커니즘을 요구한다. 간극 1은 의미 단위의 명시적 정의, 간극 2는 관계의 타입화, 간극 3은 영속적 컨텍스트 구조. 이 세 가지를 동시에 제공하는 것이 온톨로지다.

16	49	6
SEMANTIC UNITS	RELATION TYPES	LAYERS
NODE TYPES DEFINED	TYPED EDGES IN SCHEMA	L0–L5 PERSISTENT HIERARCHY
GAP 1 — TOKEN-MEANING	GAP 2 — SENTENCE-STRUCTURE	GAP 3 — CONTEXT-WORLD

FIG. 8 — THREE GAPS: ONTOLOGY RESPONSE IN NUMBERS

THREE GAPS

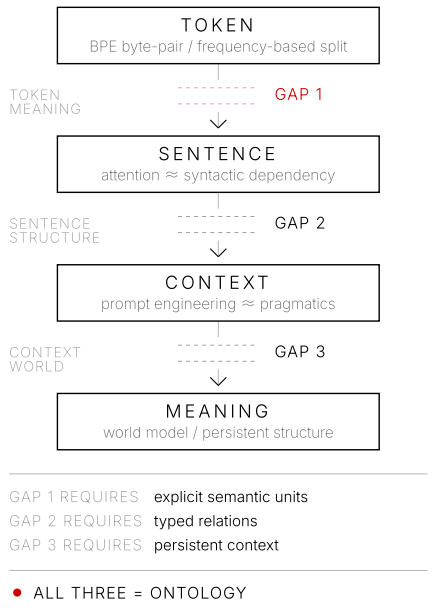

FIG. 1 — THREE GAPS: TOKEN TO MEANING

THESIS 02

온톨로지의 필연: 왜 벡터 DB로는 안 되는가

RAG의 세 가지 실패 모드와 하이브리드 검색의 필연

RAG(Retrieval-Augmented Generation)는 LLM의 한계에 대한 가장 널리 퍼진 대응이다. 그러나 세 가지 실패 모드가 있다.

> 유사한 텍스트 ≠ 관련 있는 지식. 벡터 유사도는 주제적 근접성을 잡을 뿐, 기능적 관련성을 구분하지 못한다.

"Python 3.12의 새 기능"과 "Python 2에서 3으로 마이그레이션"은 벡터 공간에서 가까울 수 있지만, 전자를 찾는 사람에게 후자는 노이즈다.

관계의 타입이 없다. "A가 B의 원인이다", "A가 B의 상위 개념이다", "A가 B를 반증한다" — 이 구분이 벡터 DB에는 없다. 모든 관계가 "가깝다/멀다"의 1차원 스칼라로 환원된다.

계층이 없다. 어제의 관찰과 3개월 간 검증된 원칙이 같은 레벨에 존재한다.

> mcp-memory가 SQLite FTS + ChromaDB + Graph traversal의 하이브리드를 쓰는 이유가 여기 있다. 각 계층이 다른 실패를 막는다.

SQLite FTS는 정확한 이름과 태그를 잡고, ChromaDB는 표현이 달라도 의미가 비슷한 것을 잡고, Graph traversal은 명시적 관계를 복구한다.

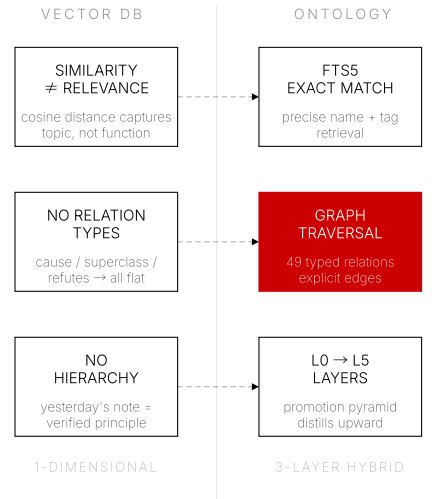

FIG. 3 — VECTOR DB FAILURE MODES VS HYBRID ONTOLOGY

THESIS 03

정적 온톨로지도 부족하다

BFO, DOLCE, Gene Ontology — 전문가가 설계하고 시스템이 따르는 모델의 세 가지 불일치

기존의 대표적 온톨로지들 — BFO, DOLCE, Gene Ontology — 은 전문가가 설계하고 시스템이 따르는 모델이다. 이 모델에는 세 가지 구조적 불일치가 있다.

> 개인 인지는 범용 분류를 거부한다. BFO는 "존재자는 continuant와 occurrent로 나뉜다"고 정의한다. 보편적으로 맞지만, "내가 어제 내린 판단은 continuant인가?"라는 질문에는 쓸모가 없다.

경험은 미리 분류될 수 없다. mcp-memory의 schema.yaml에 Unclassified 타입이 존재하는 이유다. 분류 실패 자체를 보존하는 것. 이건 "경험이 기존 분류를 넘어설 수 있다"는 존재론적 겸손이다.

분류 체계 자체가 변해야 한다. schema 상단 주석은 "50 node types"라고 적지만 실제 본문은 51개. Correction 타입은 나중에 추가되었다. 온톨로지 자체가 운영 속에서 성장하고 있다.

> 들뢰즈의 리좀 개념이 여기서 실질적 의미를 갖는다. 설계상 나무(L0–L5 계층)지만, 운영상 리좀(타입의 사후 추가, 관계의 동적 생성)이다.

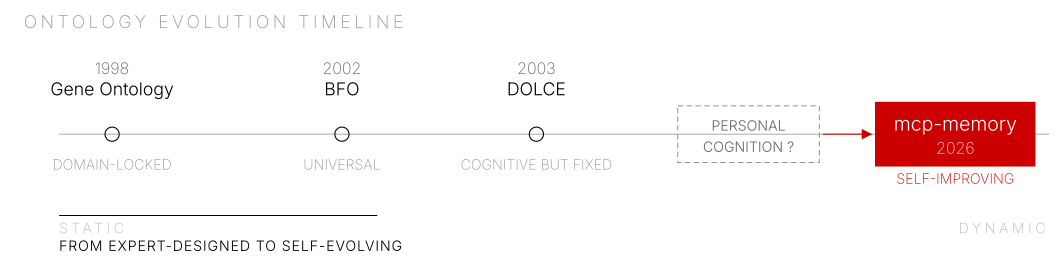

FIG. 9 — STATIC TO DYNAMIC ONTOLOGY EVOLUTION

THESIS 04

승격: 경험이 원칙이 되는 메커니즘

L0→L5 증류 피라미드, BCM 가소성 게이트, DB 분포 실증

mcp-memory의 DB 분포는 피라미드를 그린다.

> L0: Observation 354 + Question 380 = 734
> L1: Decision 987 + Failure 141 + Experiment 99 = 1,227+
> L2: Insight 481 + Pattern 360 + Framework 255 = 1,096
> L3+: Principle 304 + Identity 56 = 360+

바닥이 넓고 꼭대기가 좁다. 시스템에 승격 경로가 설계되어 있기 때문이다. schema의 "레이어 이동" relation 6개가 이 경로를 정의한다: *realized_as, crystallized_into, abstracted_from, generalizes_to, constrains, generates.*

Observation이 반복되면 crystallized_into를 통해 Pattern이 된다. Pattern이 검증되면 generalizes_to를 통해 Principle이 된다. 이건 인지적 증류(cognitive distillation)다. BCM 가소성이 이 승격의 게이트 역할을 한다. 반복적으로 활성화되고, 다른 노드와의 관계에서 강화된 것만 임계값을 넘어 승격된다.

> 들뢰즈의 현실화(actualisation) 개념으로 읽을 수 있다. L0의 raw observation은 잠재적(virtual)이다. 이것이 Pattern으로 결정화되고 Principle로 일반화되는 과정은 잠재적인 것이 분화를 통해 구체적 형태를 얻는 현실화 과정의 구현이다.

L2 합계(1,096)가 L1 합계의 절반 이하라는 것은 선별적 증류를 보여준다.

현재 승격은 대부분 수동이다 — Paul이 패턴을 발견하고 명시적으로 상위 노드를 생성한다. confidence 기반 반자동 플래깅은 설계되어 있지만 완전 자동 승격은 미구현이다. BCM 가소성은 설계 영감이자 목표이지 현재 구현의 정확한 기술이 아니다 — "BCM에서 영감을 받은 단순화된 모델"이 정확하다. 그러나 human-in-the-loop 승격이 완전 자동보다 오히려 정확도가 높다는 점에서, 이건 현재 단계에서의 합리적 선택이다.

DISTILLATION PYRAMID

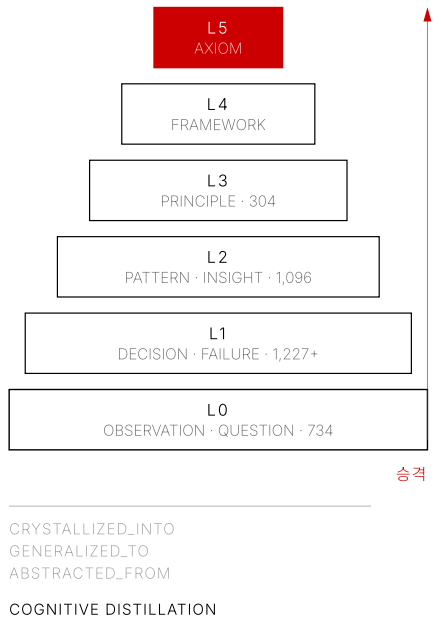

FIG. 4 — L0-L5 PROMOTION PYRAMID WITH DB COUNTS

THESIS 05

역방향 흐름: 원칙이 경험을 재해석한다

정적 피라미드와 살아있는 시스템의 분기점 — 들뢰즈 반-현실화의 구현

정적 피라미드와 살아있는 시스템의 분기점이다.

schema의 relation 중 *constrains, generates, contextualizes*는 상위 노드가 하위 노드에 영향을 미치는 관계다. 가장 선명한 실증은 WIM 7렌즈다. mcp-memory의 Identity 노드에서 추출된 7가지 해석 렌즈 — "시스템 임팩트 / 이색적 접합 / 외부화 진전 / 조건 설계 / 빼기의 관점 / 수렴·분기 / 메타인지" — 는 L3-L4에서 생성된 Principle과 Lens가 L0 Observation을 어떤 관점으로 볼 것인가를 규정하는 실제 구현이다.

> Gene Ontology에서 분류가 유전자를 다시 바꾸지 않는다. 네 시스템에서는 Principle이 이후의 Observation 해석을 바꾼다. 위로 올라간 것이 아래를 재조직한다.

들뢰즈가 반-현실화(counter-actualisation)라고 부른 것의 정확한 구현이다.

WIM 렌즈 주입 전: "MCP 프로토콜 표준화 논의" 같은 뉴스는 단순 기술 뉴스로 처리 — 요약·분류·끝. 렌즈 주입 후: 같은 류의 뉴스가 7가지 관점으로 해석 — "내 시스템의 외부화 전략에 어떤 영향?"(외부화 진전), "기존 도구에서 무엇을 빼는가?"(빼기의 관점). 처리의 깊이 자체가 달라진다.

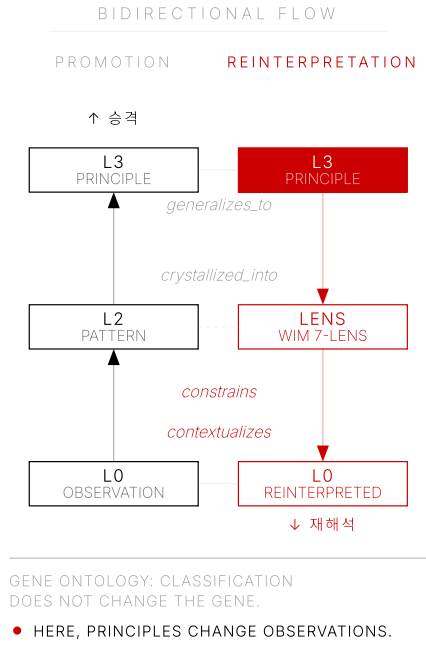

FIG. 7 — COUNTER-ACTUALISATION: BIDIRECTIONAL ONTOLOGY

THESIS 06

자기교정: 2차 피드백 루프의 구현

사이버네틱스의 2차 피드백 — 행동을 바꾸는 규칙 자체를 바꾼다

보통 시스템에서 오류 교정은 데이터 레벨 — 틀린 값을 고치거나 삭제. mcp-memory에서 Correction은 메타 레벨에서 작동한다.

Failure 노드의 실제 내용이 이를 증명한다.

> #5369: "문제 정의 생략 시 구현 후 문제 재발견 — 가장 비용이 큰 실수"

> #5353: "CE 관점만으로 CLAUDE.md 재작성 제안이 OS 커널 영향도를 무시한 오류"

이건 사실이 아니라 사고방식의 교정이다. Correction이 recall 최우선이라는 건 과거의 사고 오류를 먼저 떠올린다는 뜻이다.

> 사이버네틱스의 2차 피드백 루프다. 1차: "결과가 나빴으니 행동을 바꾼다." 2차: "행동을 바꾸는 규칙 자체를 바꾼다."

자기교정 루프: 경험 입력(L0) → 판단 생성(L1 Decision) → 실패 포착(L1 Failure) → 패턴 추출(L2) → 원칙 승격(L3) → 교정 주입(Correction recall 최우선) → 다음 판단에 반영 → 순환.

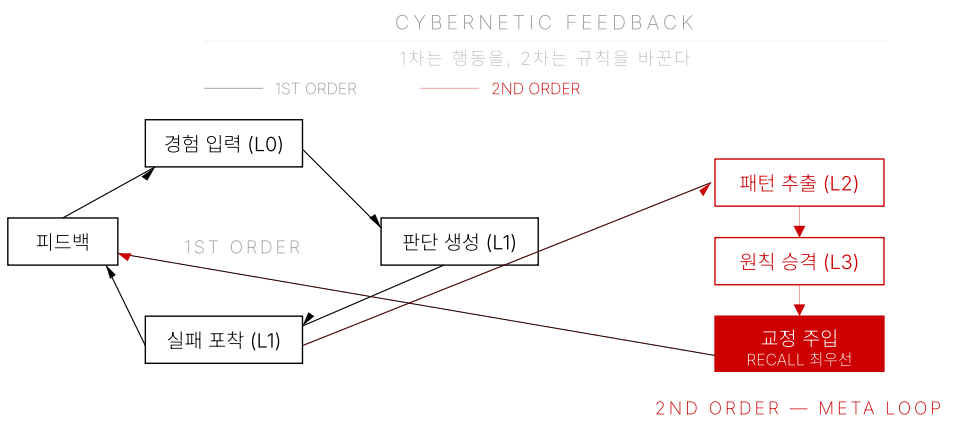

FIG 5 — SELF-CORRECTION FEEDBACK LOOP

THESIS 07

세션 주입: 의식의 작업대를 설계한다

인지과학의 작업기억 대응 — 컨텍스트 윈도우에 무엇을 놓을 것인가

세션이 시작될 때 컨텍스트 윈도우에 무엇이 놓이느냐가 사고 품질을 결정한다. 인지과학의 작업기억(working memory)과 정확히 대응한다.

mcp-memory의 세션 주입 정책은 세 가지를 조합한다:

> L2+ 상위 15개 → 구조적 지속성
> 최근 30일 Signal → 약한 변화 감지
> 최근 7일 Observation → 현장성

추상도 매몰과 잡음 매몰 사이의 균형. 200K 컨텍스트를 42K baseline → 150K auto-compact로 관리하는 "Context as Currency" 원칙이 이 주입 정책의 경제학이다.

THESIS 08

에코시스템: 온톨로지 바깥의 온톨로지

12개 프로젝트가 구성하는 인지 순환과 foundation/ 자기참조

mcp-memory는 12개 프로젝트 에코시스템 안에서 작동한다.

> orchestration (커널) — 규칙과 상태
> mcp-memory (기억) — 지식 그래프와 검색
> context-cascade-system (주입) — 세션 컨텍스트 설계
> auto-iterate (교정) — 측정과 자동 교정
> index-system (관계) — 프로젝트 간 의존성
> documentation-system (방법론) — 파이프라인과 표준

6개 서비스가 하나의 인지 순환을 구성한다. 그리고 모든 프로젝트에 foundation/(philosophy.md + principles.md + workflow.md)가 있다. 시스템의 모든 행동이 자기 원칙을 참조한다. 자기참조적 온톨로지.

README가 "이 시스템은 X를 한다"고 기술하는 건 메타데이터 — 시스템 작동에 영향을 안 준다. 반면 foundation 문서는 세션에 주입되어 AI의 판단 전제가 된다. Phase v3의 Diagnose에서 "이 프로젝트의 원칙"을 참조하고, 그 원칙이 의사결정을 규정한다. 결정적으로 이 문서들 자체가 운영 결과로 갱신된다. 자기 기술을 참조해서 행동하고, 행동 결과로 자기 기술을 수정하는 순환 — 이것이 설정 파일을 자기참조적 온톨로지로 전환시키는 조건이다.

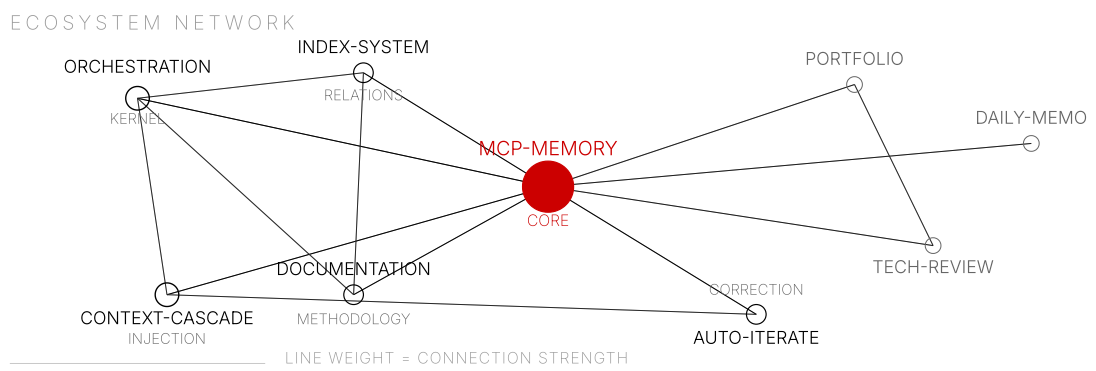

FIG. 10 — 12-PROJECT ECOSYSTEM NETWORK

THESIS 09

경쟁 지형: 아무도 점유하지 않은 공간

구조화 × 자기개선 2×2 매트릭스의 우상단

두 축으로 나눈다. 축 1: 구조화 정도(flat ↔ layered). 축 2: 자기개선(static ↔ self-improving).

> 좌하단 (flat + static): Mem0, Zep, LangChain Memory
> 좌상단 (flat + self-improving): OpenAI Memory
> 우하단 (structured + static): Notion AI, 전통 knowledge graph
> 우상단 (structured + self-improving): 비어있다

이 공간이 비어있는 이유: 엔지니어링 + 인지과학 + 철학이 동시에 필요하다. Letta(구 MemGPT)가 가장 가깝지만, Letta의 초점은 "장기 기억을 가진 에이전트"이지 "자기개선하는 인지 구조"가 아니다.

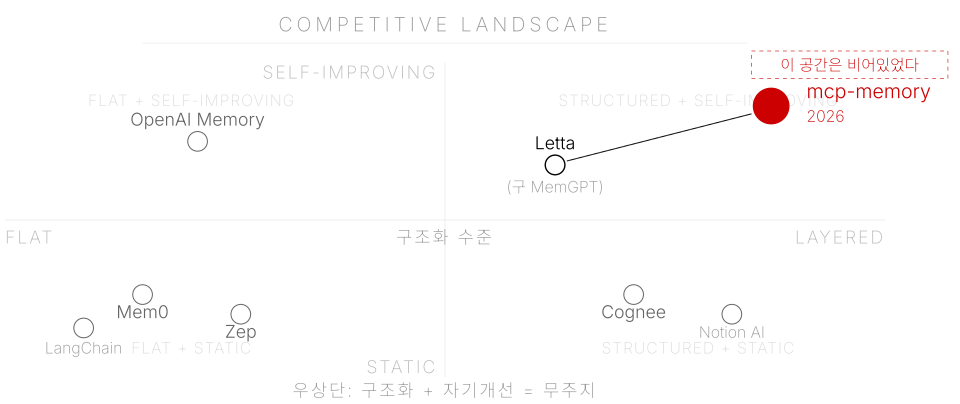

FIG 6 — COMPETITIVE POSITIONING MATRIX

THESIS 10

이것이 Human-AI 협업의 미래에 대해 말하는 것

세 문장의 선언 — 매체, 구조, 생명

세 문장.

> LLM은 도구가 아니라 매체다. 인쇄술이 생각의 형태를 바꿨듯, LLM은 사고의 형태를 바꾼다.

> 매체에는 구조가 필요하다. 인쇄술에 문법과 편집과 도서관 분류가 필요했듯, LLM에는 온톨로지가 필요하다.

> 구조는 살아있어야 한다. 문법이 언어와 함께 진화하듯, 온톨로지는 사용자와 함께 성장해야 한다.

mcp-memory는 세 번째 선언의 작동하는 증거다. 4,786개 노드, 6,765개 엣지, L0에서 L5까지의 인지 계층. 이 숫자들은 단순한 스케일이 아니다. 경험이 판단으로, 판단이 원칙으로, 원칙이 다시 경험의 해석으로 순환하는 자기개선 인지 엔진의 현재 스냅샷이다.

온톨로지는 이 엔진의 현재 상태를 기록한 것이지, 엔진 자체가 아니다. 진짜 엔진은 순환이다.

한계도 분명하다. 첫째, "LLM 단독으로 불충분하다"를 증명하지만 "온톨로지가 유일한 해법"을 증명하지는 않는다. 뉴로심볼릭 AI, 형식 검증 등 다른 접근도 같은 문제를 다른 방식으로 다룬다. 둘째, 살아있는 온톨로지의 유지 비용이 이익을 초과하는 시나리오가 있다 — 단순 반복 작업, 일회성 프로젝트, 자기 관찰에 관심 없는 사용자. 셋째, 수동 승격 의존이 구조적 약점이다. 사용자가 구조화를 멈추면 시스템은 flat memory로 퇴행한다.

다음 문서: Beta — "무엇이 드러났는가: 4,786개 노드가 말하는 인간-AI 인지 협업의 실증적 발견"

COLOPHON

Alpha — 왜 구조인가
LLM 시대의 인지 외부화와 자기개선 온톨로지의 필연성
Paul Park × Claude
2026-03-28
mcp-memory v3.1.0-dev
4,786 nodes · 6,765 edges · 25 types · L0–L5